UNIBALSAS Centro Universitário

ANÁLISE E DESENVOLVIMENTO DE SISTEMAS

Análise e Desenvolvimento de Sistemas

Unidade Curricular: ALGORITMOS E PROGRAMAÇÃO AVANÇADA

CENTRO UNIVERSITÁRIO UNIBALSAS

RELATÓRIO TÉCNICO - TED 03

GERENCIADOR DE PERSISTÊNCIA DE DADOS EM ARQUIVOS ESTRUTURADOS

(CSV E JSON)

GUSTAVO SILVA DO NASCIMENTO - RA 26.1.15196

KAUÊ LIMA OLIVEIRA - RA 26.1.19982

BALSAS - MA

2026

1 IDENTIFICAÇÃO

Instituição	Centro Universitário UNIBALSAS
Curso	Análise e Desenvolvimento de Sistemas
Disciplina	Algoritmos e Programação Avançada
Atividade	TED 03 - Gerenciador de Persistência de Dados em Arquivos Estruturados (CSV e JSON)
Integrantes	Gustavo Silva do Nascimento (RA 26.1.15196); Kauê Lima Oliveira (RA 26.1.19982)
Tecnologia	Python 3.10 ou superior
Modalidade	Em dupla
Prazo da atividade	31/10/2026

2 INTRODUÇÃO E OBJETIVOS

A persistência de dados permite que informações sejam mantidas após a execução de um programa. Nesta atividade, foi implementada uma aplicação de linha de comando em Python para ler, escrever, converter e atualizar dados nos formatos CSV (Comma-Separated Values) e JSON (JavaScript Object Notation). O projeto se concentra na integridade dos registros, no fechamento seguro de arquivos e no tratamento de falhas de entrada e saída.

O objetivo geral consiste em desenvolver um módulo capaz de manipular arquivos estruturados com conversão bidirecional e segurança operacional. São objetivos específicos:

- realizar leitura e gravação de arquivos CSV e JSON com codificação UTF-8;
- converter entre formatos e preservar dados, inclusive acentos, zeros à esquerda e tipos de campos JSON quando houver metadados;
- empregar **with open(...)** para fechamento automático de arquivos;
- tratar erros de arquivo inexistente, falta de permissão, codificação e estrutura inválida;
- produzir backup antes de operações que substituam conteúdo existente;
- comprovar o funcionamento por testes automatizados e evidências de execução.

3 DESCRIÇÃO DA SOLUÇÃO

A solução utiliza módulos da biblioteca padrão do Python, principalmente **csv**, **json**, **pathlib**, **hashlib** e **os**. A interface está em **main.py**; a leitura, a gravação, o backup e a escrita atômica ficam em **src/persistencia/arquivos.py**; as conversões e atualizações são implementadas em **src/persistencia/servico.py**.

3.1 Organização dos arquivos

```
TED03_Gerenciador_Persistencia/
  main.py
  src/persistencia/{__init__.py, arquivos.py, servico.py}
  dados/{clientes.csv, produtos.json}
  tests/test_persistencia.py
  evidencias/{cenario_1...4.png, cenario_1...4.txt, ...}
  relatorio/TED03_Relatorio_Gustavo_Kaue.pdf
  README.md
```


Km 5 da BR 230, Fazenda Malidere IV
65800-000, Balsas-MA
99 3542-5500 - unibalsas. edu.br

3.2 Fluxo de execução

A aplicação recebe um comando, valida os argumentos e aciona a função responsável. A conversão de CSV para JSON lê os registros como textos, preservando os valores literais. A conversão de JSON para CSV transforma os campos em células e gera um arquivo auxiliar **.tipos.json** para recuperar tipos JSON na conversão de retorno. O arquivo auxiliar também armazena o hash SHA-256 do CSV para detectar modificações externas.

Na atualização de arquivo existente, o conteúdo anterior é salvo em arquivo **.bak** com identificador único, seguido por gravação em arquivo temporário e substituição do destino. Se a criação do backup falhar, a alteração não prossegue. A interface exibe mensagens com os prefixos [OK], [INFO], [BACKUP] e [ERRO].

4 IMPLEMENTAÇÃO

4.1 Leitura segura com gerenciador de contexto

A utilização de **with open(...)** garante o fechamento do recurso quando a operação termina, mesmo se surgir exceção. O modo de leitura CSV utiliza **utf-8-sig** para aceitar arquivos UTF-8 com ou sem marca de ordem de bytes (BOM), além de **newline=""** para compatibilidade com o módulo csv.

```
def ler_json(caminho: Path):
    caminho = Path(caminho)
    with open(caminho, "r", encoding="utf-8-sig") as arquivo:
        def rejeitar_constante(valor):
            raise EstruturaInvalida(f"Número JSON não permitido: {valor}")
        return json.load(arquivo, parse_constant=rejeitar_constante)
```

4.2 Backup e escrita segura

Antes da substituição de um destino existente, a rotina **criar_backup** copia o arquivo para um novo nome terminado em **.bak**. Os arquivos temporários evitam gravar uma atualização diretamente sobre o conteúdo original. O trecho a seguir apresenta a abertura segura para gerar o backup:

```
with open(caminho, "rb") as origem, open(destino, "xb") as copia:
    shutil.copyfileobj(origem, copia)
print(f"[BACKUP] Cópia anterior preservada em: {destino}")
```

Na gravação, o fluxo é: verificar/criar backup → escrever arquivo temporário → executar **os.replace(temporario, caminho)**. Esse processo limita o risco de deixar o arquivo principal parcialmente escrito em falhas durante a escrita.

4.3 Conversão CSV/JSON e preservação de valores

A leitura CSV usa **csv.DictReader**; a gravação usa **csv.DictWriter**. Na conversão de JSON para CSV, um mapa de tipos distingue strings, inteiros, decimais, booleanos, listas, objetos, valores nulos e chaves ausentes. No retorno para JSON, esse mapa permite recuperar a estrutura original. Sem metadados, o conteúdo CSV continua sendo tratado como texto, evitando inferência ambígua de tipos.

```
for coluna in colunas:
    if coluna not in registro:
        linha[coluna], tipo_linha[coluna] = "", "ausente"
    else:
        linha[coluna], tipo_linha[coluna] = _serializar_celula(
            registro[coluna]
        )
```


Km 5 da BR 230, Fazenda Malidere IV
65800-000, Balsas-MA
99 3542-5500 - unibalsas. edu.br

4.4 Tratamento de exceções e mensagens

A interface principal diferencia falhas comuns para orientar o usuário sem exibir traceback inesperado:

```
except FileNotFoundError as exc:
    print(f"[ERRO] Arquivo não encontrado: {exc.filename}",
          file=sys.stderr)
    return 1
except PermissionError as exc:
    print(f"[ERRO] Permissão negada ao acessar: {exc.filename}",
          file=sys.stderr)
    return 1
except UnicodeError:
    print("[ERRO] Codificação incompatível: utilize arquivos UTF-8.",
          file=sys.stderr)
    return 1
```

5 TESTES E EVIDÊNCIAS DE EXECUÇÃO

Os testes foram executados em ambiente Python, a partir da raiz do projeto, usando arquivos de amostra e arquivos temporários. As quatro figuras seguintes são capturas de tela do terminal durante a execução dos comandos; os registros textuais originais estão anexados em **evidencias/*.txt**, e os arquivos resultantes podem ser conferidos em **evidencias/arquivos_gerados**.

5.1 Cenário 1 - Conversão de CSV para JSON

O arquivo **dados/clientes.csv** foi lido e convertido. O JSON gerado apresenta três registros com nomes acentuados e valores de identificação preservados como texto. O teste automatizado também confirma que o identificador 001 não se converte incorretamente no número 1.

```
$ python main.py csv-para-json dados/clientes.csv evidencias/arquivos_gerados/clientes.json
[BACKUP] Cópia anterior preservada em: evidencias/arquivos_gerados/clientes.json.20261101T021005Z.d3a
5e180.bak
[OK] CSV convertido para JSON: evidencias/arquivos_gerados/clientes.json (3 registro(s)).
[INFO] CSV não possui metadados; os valores foram preservados como texto.
$ head -n 15 evidencias/arquivos_gerados/clientes.json
[
  {
    "id": "001",
    "nome": "Ana Vitória",
    "cidade": "Balsas",
    "telefone": "099988776655"
  },
  {
    "id": "002",
    "nome": "João Pedro",
    "cidade": "Imperatriz",
    "telefone": "099977665544"
  },
  {
    "id": "003",
```

Figura 1 - Captura da execução de CSV → JSON e conferência dos dados convertidos.

5.2 Cenário 2 - Conversão de JSON para CSV e retorno

A amostra **dados/produtos.json** foi convertida para CSV, com geração do arquivo auxiliar de tipos. Na conversão de retorno para JSON, a comparação programática das estruturas original e recuperada indicou igualdade, abrangendo preço numérico, valores booleanos, lista de variantes e valor nulo.

Km 5 da BR 230, Fazenda Malidere IV
65800-000, Balsas-MA
99 3542-5500 - unibalsas. edu.br

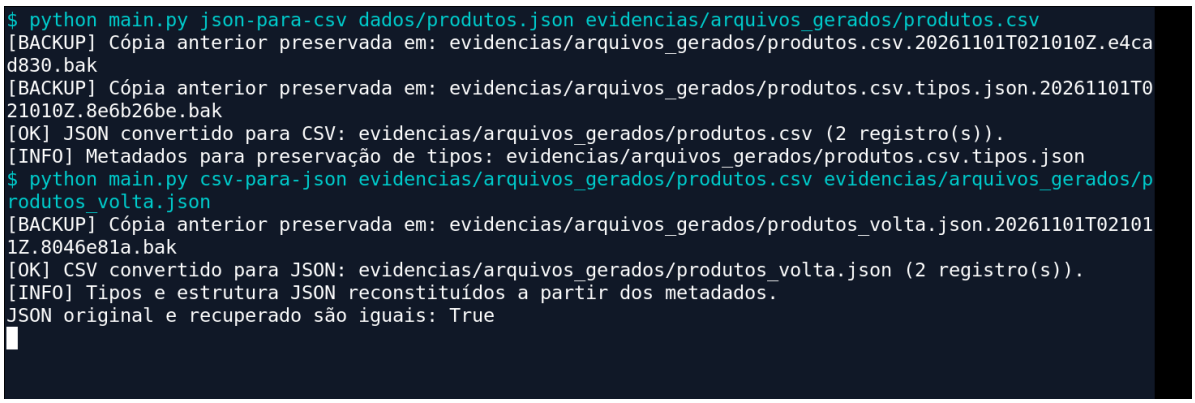


Figura 2 - Captura da conversão JSON → CSV → JSON com validação de igualdade.

5.3 Cenário 3 - Atualização com backup prévio

Um arquivo JSON anteriormente criado foi atualizado no campo cidade do primeiro registro. Antes da alteração, a aplicação criou um backup contendo o valor anterior, **Balsas**. Após a atualização, o arquivo principal passou a conter **Imperatriz**, sem eliminar a cópia original.

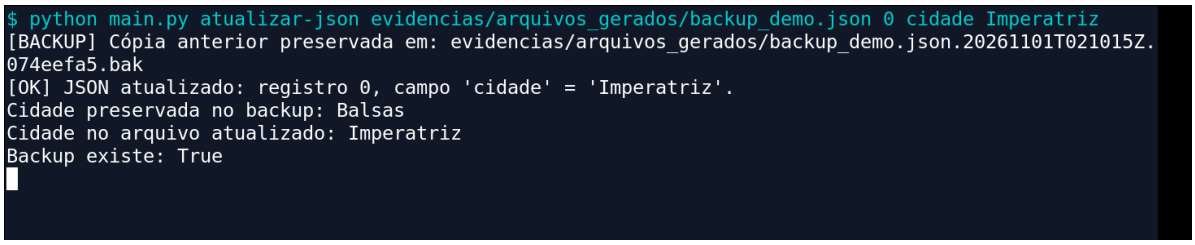


Figura 3 - Captura da criação de backup antes da atualização e conferência dos valores.

5.4 Cenário 4 - Tratamento de erro de arquivo

Foi solicitada a leitura de um arquivo que não existe. A aplicação produziu uma mensagem de erro compreensível e retornou código 1, sem encerrar por exceção não tratada.

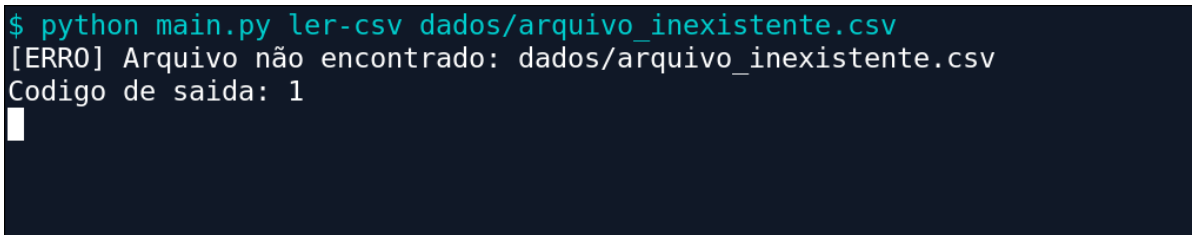


Figura 4 - Captura do tratamento da exceção FileNotFoundError.

5.5 Resultado dos testes automatizados

Foi executado o comando **python -m unittest discover -s tests -v**. O conjunto contém **13 testes**, todos com resultado **OK**, incluindo conversões, metadados, cópias de segurança e falhas simuladas de entrada. O log completo está disponível em **evidencias/testes_automatizados.txt**.

Km 5 da BR 230, Fazenda Maliderе IV
65800-000, Balsas-MA
99 3542-5500 - unibalsas. edu.br

Grupo de testes	Resultado observado
Conversão CSV → JSON	Preserva textos, acentos e zeros à esquerda
Conversão JSON → CSV → JSON	Preserva tipos, campos ausentes e estrutura original
Backup antes da alteração	Cópia anterior preservada em CSV/JSON e saídas existentes
Atualização com metadados	Atualiza soma de verificação e tipo da célula
Erros e validações	Arquivo inexistente, codificação, colunas e JSON inválidos

6 CONCLUSÃO

A aplicação atendeu aos objetivos centrais de persistência de dados estruturados em CSV e JSON. As conversões funcionaram nos dois sentidos, e o mecanismo auxiliar de metadados demonstrou a preservação dos tipos JSON na conversão de ida e volta. A utilização de gerenciadores de contexto, gravação temporária e backup anterior à escrita aumentou a segurança das operações. O conjunto de treze testes terminou sem falhas, e as evidências registram tanto situações de funcionamento esperado quanto uma falha controlada.

Entre os desafios técnicos, destaca-se a diferença entre a tipagem de dados em JSON e a natureza textual das células CSV. Esse problema foi tratado com arquivo auxiliar de metadados e hash de integridade. Como limitação, a atualização de CSV e de seu arquivo auxiliar utiliza operações de gravação separadas; portanto, em situações excepcionais de falha entre as duas escritas, pode ser necessário recuperar os backups correspondentes.

Os arquivos de código, amostras, testes, evidências e o presente relatório integram o repositório e permitem reproduzir os procedimentos descritos.

7 REFERÊNCIAS

PYTHON SOFTWARE FOUNDATION. *csv — CSV File Reading and Writing*. Disponível em: https://docs.python.org/3/library/csv.html. Acesso em: 31 out. 2026.

PYTHON SOFTWARE FOUNDATION. *json — JSON encoder and decoder*. Disponível em: https://docs.python.org/3/library/json.html. Acesso em: 31 out. 2026.

PYTHON SOFTWARE FOUNDATION. *Built-in Functions: open()*. Disponível em: https://docs.python.org/3/library/functions.html#open. Acesso em: 31 out. 2026.

PYTHON SOFTWARE FOUNDATION. *os — Miscellaneous operating system interfaces: os.replace()*. Disponível em: https://docs.python.org/3/library/os.html#os.replace. Acesso em: 31 out. 2026.

UNIBALSAS. *TED 03 - Gerenciador de Persistência de Dados em Arquivos Estruturados (CSV e JSON)*. Enunciado da atividade, 2026.

Km 5 da BR 230, Fazenda Maliderе IV
65800-000, Balsas-MA
99 3542-5500 - unibalsas. edu.br